

Système de stockage distribué basé sur une DHT

Projet Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Auteurs: Cedric Genevieve, Safa Gunes, Somaya Boudalil
Étudiants en Master 2026 — Systèmes et Applications Répartis

Instructeur: Prof. Alessio Pagliari

Date: 27 avril 2026

Table des matières

1	Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué	3
1.1	Étude des approches P2P structurée: Les DHTs	3
1.1.1	Introduction au problème du Lookup	3
1.1.2	Comparaison des stratégies de recherche	4
1.1.3	Fondamentaux des Distributed Hash Tables	6
1.1.4	Mécanismes des DHTs	7
1.1.5	Interfaces des DHTs	7
1.1.6	Récapitulatif	8
1.2	Conclusion et fil conducteur du projet	8
2	Première génération : architecture centralisée inspirée de Napster	9
2.1	Vue d'ensemble de l'architecture	9
2.2	Fonctionnement des opérations fondamentales	9
2.2.1	Opération PUT	9
2.2.2	Opération GET	10
2.2.3	Opération Delete	10
2.3	Gestion dynamique des nœuds	11
2.3.1	Intégration d'un nouveau nœud (JOIN)	11
2.3.2	Retrait d'un nœud (LEAVE)	11
2.3.3	Détection des pannes par heartbeat	12
2.4	Analyse des performances	12
2.5	Avantages et limites de l'approche centralisée	12
3	Deuxième génération : architecture non structurée inspirée de Gnutella	13
3.1	Positionnement général et lien avec Gnutella	13
3.2	Problème initial : l'absence de vue globale	13
3.3	Passage à un mesh : ce que cela corrige, et ce que cela casse	14
3.4	Le flooding comme correction partielle	16
3.5	Réponse directe, puis <i>reverse path caching</i>	18
3.6	Usages réels et intérêt pratique	19
3.7	Bilan	20
4	Troisième génération : DHT structurée inspirée de Chord	21
4.1	Vue d'ensemble de l'architecture	21
4.1.1	Principe de fonctionnement	21
4.1.2	Routage efficace : Finger Table	21
4.1.3	Opération PUT	21
4.1.4	Opération GET	22
4.1.5	Intégration d'un nouveau nœud (JOIN)	22
4.1.6	Retrait d'un nœud (LEAVE)	22
4.1.7	Détection des pannes et stabilisation	22
4.1.8	Avantages et limites de l'approche Chord	22

1 Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué

La croissance du volume de données a rendu les architectures centralisées insuffisantes. Les systèmes distribués sont devenus la norme, mais posent un défi central : garantir un *lookup* efficace, la *consistency* et la *fault tolerance* dans un environnement de nœuds peu fiables.

Dans ce projet Master, nous explorons l'évolution des architectures de stockage distribué à travers trois générations successives, chacune répondant aux limites de la précédente :

1. L'architecture centralisée (inspirée de Napster) Un serveur central coordonne l'ensemble du réseau, mais cette approche reste vulnérable à un point de défaillance unique (*single point of failure*).

2. L'architecture pair-à-pair non structurée (inspirée de Gnutella) Elle supprime la dépendance au serveur central au profit d'un modèle P2P (architecture *symmetric*) où chaque nœud est à la fois client et serveur, au prix d'un *overhead* de communication élevé et de résultats non garantis.

3. L'architecture DHT structurée (inspirée de Chord) Elle structure l'*overlay* pour garantir un routage déterministe en $O(\log N)$, offrant le meilleur compromis entre scalabilité, résilience au *churn* et coût de maintenance.

1.1 Étude des approches P2P structurée: Les DHTs

Cette section résume les principes essentiels des *Distributed Hash Tables* (DHTs) : localisation des données, routage, gestion des nœuds et interfaces applicatives.

Référence : WEHRLE, GÖTZ et RIECHE (2005).

1.1.1 Introduction au problème du Lookup

Le problème central est le *lookup problem* : où stocker une donnée D , puis comment la retrouver dans un réseau distribué sans coordination centrale.

"Où stocker, et comment trouver un certain élément de donnée D dans un système distribué sans aucun contrôle ou coordination centralisés ?"

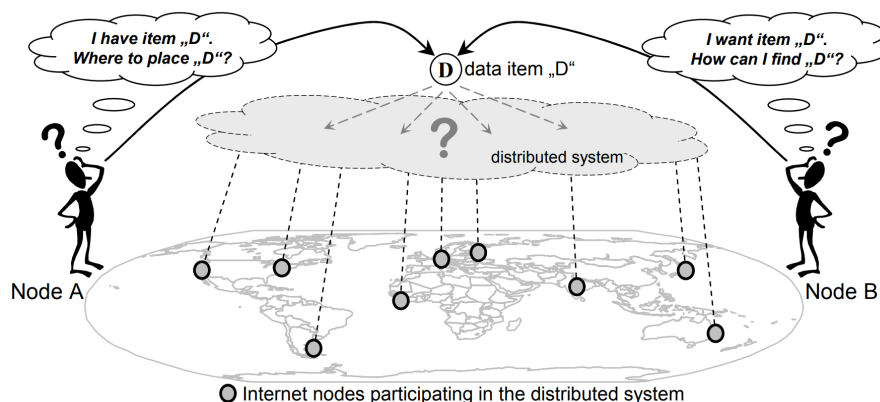


FIGURE 1 – **Le problème du lookup** : le nœud A stocke une donnée D et le nœud B doit la retrouver sans connaître son emplacement. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.2 Comparaison des stratégies de recherche

Trois familles de solutions sont comparées.

1. Central Server

Approche de **première génération** (ex. Napster). Un serveur central indexe les contenus. Recherche rapide en $O(1)$, mais dépendance forte à un *single point of failure*.

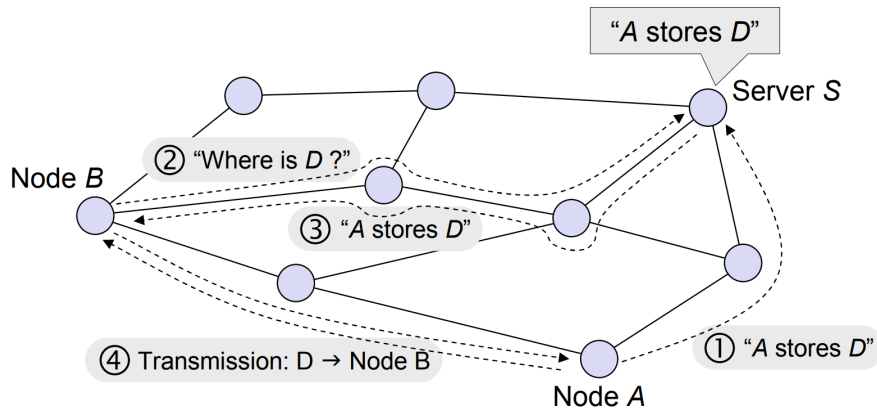


FIGURE 2 – **Central Server** : publication, requête au serveur central, puis téléchargement direct entre pairs. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

2. Flooding Search

Approche de **deuxième génération** (ex. Gnutella). La requête est propagée de voisin en voisin via un *reactive flooding* jusqu'au *Time To Live* (TTL). L'approche évite le serveur central, mais génère un fort *communication overhead* et des résultats non garantis.

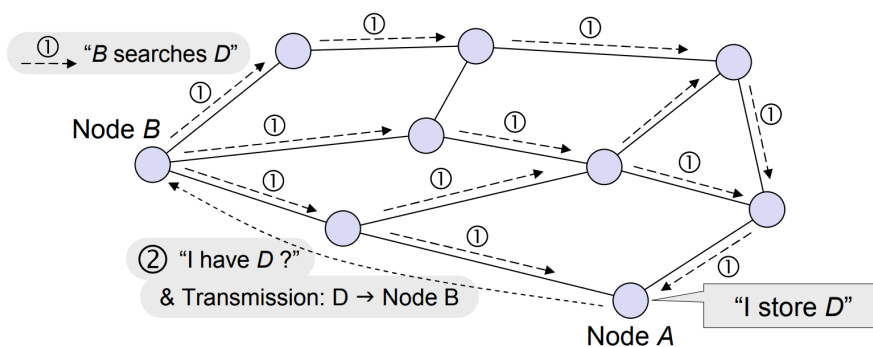


FIGURE 3 – **Flooding Search** : diffusion récursive de requêtes, puis retour des nœuds ayant la donnée. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

3. Distributed Hash Tables (DHTs)

Les DHTs (*structured Peer-to-Peer systems*) structurent l'*overlay* pour obtenir un compromis robuste : $O(\log N)$ entrées de routage par nœud et $O(\log N)$ *hops* pour localiser une donnée.

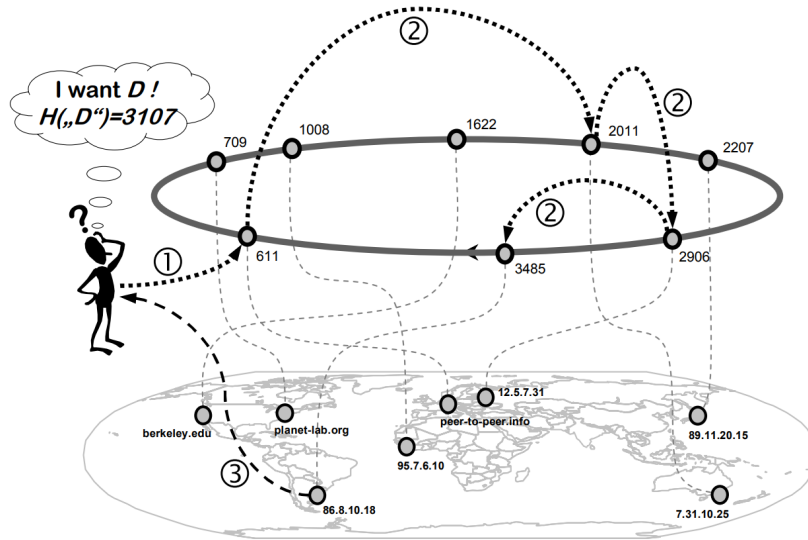


FIGURE 4 – **Distributed Hash Table** : routage structuré et efficace vers le nœud responsable. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Les Figures 2, 3, 4 et 5 montrent que les DHTs offrent le meilleur compromis entre coût de recherche et coût de stockage.

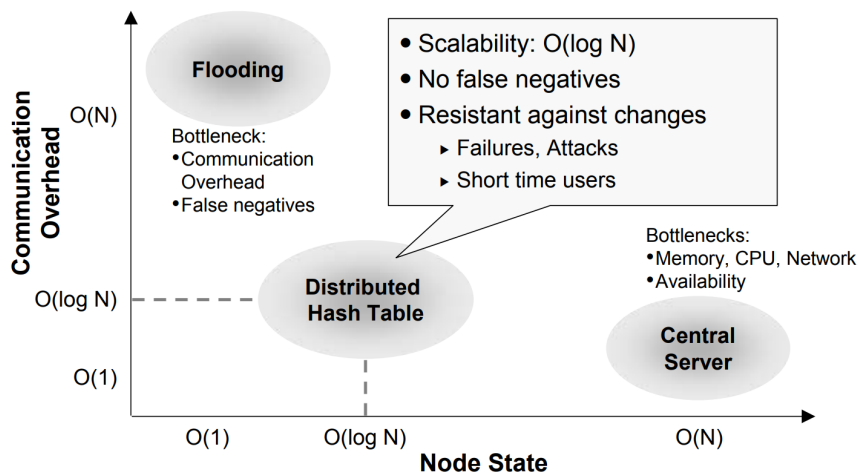


FIGURE 5 – Comparaison de la complexité en termes d'effort de recherche (axe y) et de coût de stockage par nœud (axe x). Les goulots d'étranglement (*bottlenecks*) ainsi que les caractéristiques spécifiques de chaque approche sont indiqués. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.3 Fondamentaux des Distributed Hash Tables

Une DHT associe chaque clé à un nœud responsable via un espace d'adressage logique et un routage déterministe.

Gestion distribuée des données

Chaque nœud gère une plage de clés et conserve une *partial view* du système. Les messages sont relayés de nœud en nœud jusqu'à la destination.

Adressage dans les Distributed Hash Tables

Nœuds et données partagent le même *address space* logique (souvent très grand, p. ex. 0 à $2^{160} - 1$). Les IDs sont produits par une *collision-resistant hash function*.

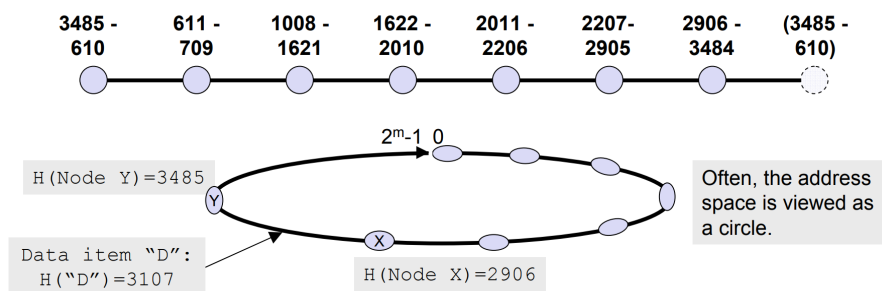


FIGURE 6 – Espace d'adressage linéaire partitionné entre paires. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

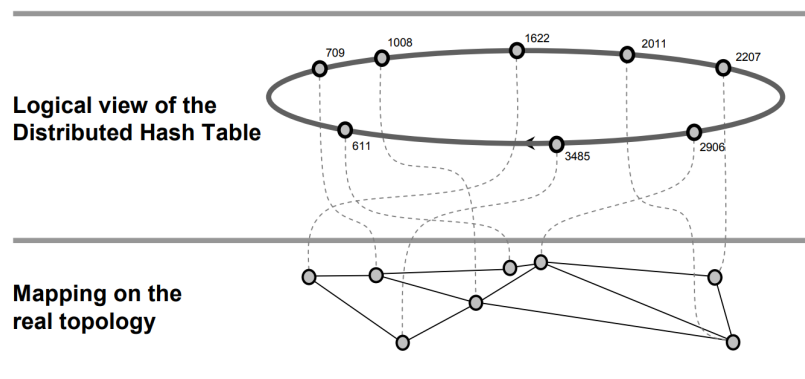


FIGURE 7 – Vue logique de la DHT et vue du réseau sous-jacent. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Routage

Chaque nœud transmet vers un voisin plus proche de l'ID cible, ce qui réduit progressivement la distance jusqu'au nœud responsable en un *small number of hops*.

Méthodes de stockage

Direct Storage : la donnée est copiée sur le nœud responsable (meilleure *availability*, coût plus élevé).

Indirect Storage : la DHT stocke un *pointer* vers la donnée (coût réduit, dépendance au nœud source).

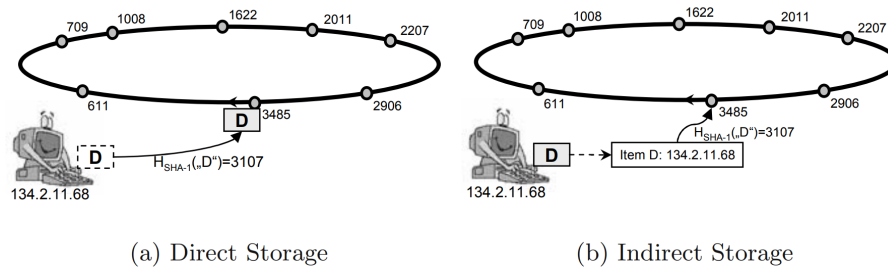


FIGURE 8 – Comparaison entre stockage direct et stockage indirect. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

1.1.4 Mécanismes des DHTs

La robustesse dépend de la gestion du *churn*, c'est-à-dire le taux de *joins* et *leaves* (arrivées/départs) ainsi que les pannes fréquentes.

Arrivée de nœuds

L'entrée d'un nœud suit quatre étapes courtes :

1. **Bootstrap** : le nouveau pair contacte un nœud existant qui sert d'*entry point*.
2. **Address Assignment** : il reçoit sa partition dans le *logical address space*.
3. **Routing Update** : les tables de routage sont ajustées pour intégrer le nouveau pair.
4. **Data Transfer** : les paires (*key*, *value*) devenues sa responsabilité lui sont transférées.

Défaillance de nœuds

Deux approches coexistent : *reactive recovery* (détection pendant le routage, puis route alternative) et *proactive recovery* (*probes* périodiques, remplacement préventif des entrées invalides). La *replication* limite la perte de données applicatives.

Départ de nœuds

Un départ volontaire (*graceful leave*) facilite la migration immédiate des données vers le *successor* et accélère la convergence des tables de routage. Il améliore aussi la *routing efficiency* et stabilise les mécanismes de *replication* et de *load-balancing*.

1.1.5 Interfaces des DHTs

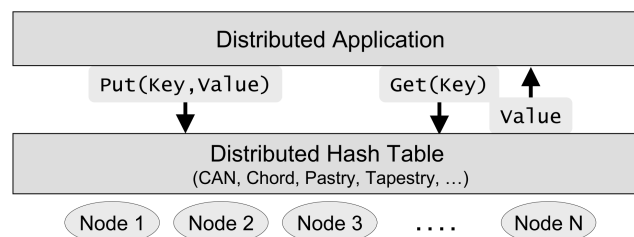


FIGURE 9 – Interface d'une Distributed Hash Table, typiquement put/get. [Source:schéma pris des notes de cours WEHRLE, GÖTZ et RIECHE 2005]

Interface de routage

Interface *stateless* orientée messages : *send* livre vers un ID destination, *receive* remet les messages à l'application (*delivery semantics*). Cette *stateless interface* se concentre sur la livraison de paquets basés sur un ID logique plutôt que sur une adresse IP physique.

Interface de stockage

Interface clé-valeur : *put* écrit la donnée, *get* la récupère. La couche gère *routing failures*, *replication* et *admission control*. Cette couche ajoute de la complexité car elle gère la persistance et la fiabilité des données, comme une table de *hash* distribuée.

Interface Client

Des hôtes peuvent consommer le service DHT sans participer au routage de l'*overlay* (mode *client*). Ce mode est utile quand on veut séparer usage applicatif et participation au routage (fiabilité, politiques d'*access control*).

1.1.6 Récapitulatif

Les DHTs offrent une excellente *scalability* sans point central, mais imposent des compromis entre efficacité de routage, coût de maintenance et résilience au *churn*.

Défis de conception

- **Routing Efficiency** : qualité du routage selon la structure des tables et la topologie.
- **Management Overhead** : coût de maintenance en continu.
- **Churn** : impact des arrivées/départs simultanés sur la stabilité.

Limitations fondamentales

- **Untrusted Environments** : résistance difficile face aux nœuds malveillants et aux *Byzantine faults*.
- **Query Constraints** : *fuzzy searches* et requêtes textuelles complexes peu adaptées au modèle par identifiants.

1.2 Conclusion et fil conducteur du projet

Ces trois générations forment une progression naturelle, chacune répondant aux limitations précises de la précédente. Pour en saisir pleinement les enjeux, nous allons les étudier chacune en détail.

Notre progression dans le document Nous commencerons par l'architecture centralisée inspirée de Napster, dont la simplicité sert de point de départ et de référence.

Nous verrons ensuite comment l'architecture non structurée inspirée de Gnutella tente de s'affranchir du point central de défaillance, et quelles nouvelles limitations elle introduit.

Enfin, nous présenterons l'architecture DHT structurée inspirée de Chord, qui résout les problèmes fondamentaux de placement et de routage identifiés dans les deux générations précédentes.

Ouverture vers l'évaluation Pour clore cette étude, une section dédiée aux mesures de performances mettra en regard les trois architectures, afin de mettre en lumière leurs différences concrètes en terme de scalabilité et rapidité.

2 Première génération : architecture centralisée inspirée de Napster

2.1 Vue d'ensemble de l'architecture

Cette première implémentation adopte une architecture centralisée inspirée de *Napster*. L'objectif est de proposer un système de stockage clé-valeur simple à raisonner, dans lequel un composant central conserve une vue globale des nœuds actifs et décide du placement des données. Cette approche sert de point de départ pour comparer ensuite ses limites avec des architectures plus distribuées.

Cette première architecture repose sur deux types de composants :

- Un coordinator : composant unique jouant le rôle de point d'entrée pour toutes les requêtes clientes. Il calcule le nœud responsable d'une clé et relaie les opérations vers celui-ci.
- Des nœuds de stockage : Connu et accessible uniquement par le coordinator, sert à stocker les données des clients.

La distribution des données repose sur une fonction de hachage modulaire : pour une clé donnée, le nœud responsable est identifié par $\text{hash}(\text{key}) \% N$, où N désigne le nombre courant de nœuds actifs. Mathématiquement, si l'on note (k) la clé et (N) le nombre de nœuds actifs, alors le nœud responsable est donné par :

$$\text{node}(k) = h(k) \bmod N$$

L'ensemble des communications entre les composants s'effectue via des sockets TCP. Chaque message est sérialisé en Java et typé, encapsulant toutes les informations nécessaires au traitement de la requête : clé, valeur, source et compteur de sauts. Côté nœud de stockage, chaque connexion entrante est traitée dans un thread dédié créé dynamiquement, ce qui permet au nœud de gérer plusieurs requêtes simultanées sans blocage.

2.2 Fonctionnement des opérations fondamentales

2.2.1 Opération PUT

La Figure 10 illustre le déroulement d'une requête d'écriture. Lorsque le client 1 souhaite insérer la paire (40,25), il envoie un message `CLIENT_PUT` au coordinator. Ce dernier applique la fonction de hachage modulaire $\text{hash}(40) \% 5 = 0$ ce qui désigne le nœud 8001 comme unique responsable de cette clé. Le coordinator transmet alors un message `NODE_PUT` directement à ce nœud. Le nœud 8001 stocke la paire localement et répond par un `ACK`. Le coordinator retransmet enfin une `CLIENT_RESPONSE(OK)` au client pour confirmer le succès de l'opération. On observe que les nœuds 8002 à 8005, représentés en tirets, ne sont à aucun moment sollicités.

À noter que si aucun nœud actif n'est disponible au moment de la requête, le coordinator retourne directement une erreur au client.

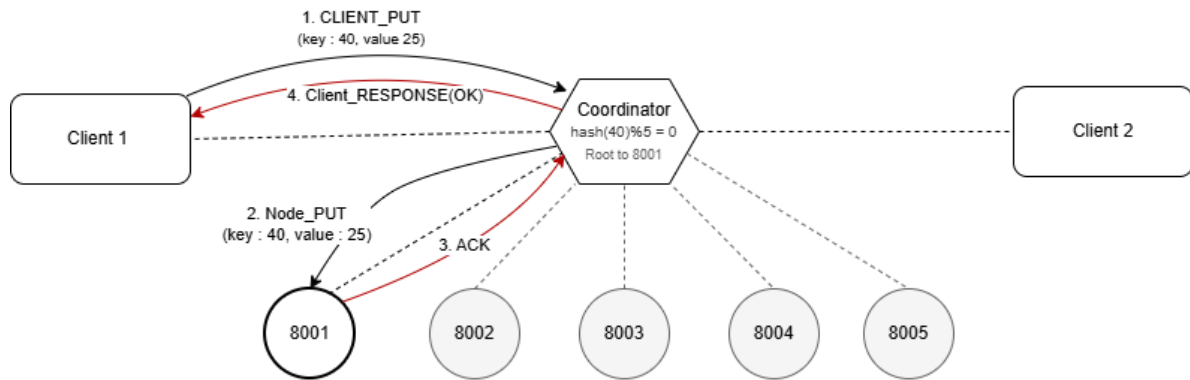


FIGURE 10 – Déroulement d'une opération PUT dans l'architecture centralisée.

Nombre total de message échangé au sein du réseau = 4 (client → coordinator → node → coordinator → client)

2.2.2 Opération GET

La Figure 11 illustre le déroulement d'une requête de lecture. Le client 2 envoie un CLIENT_GET portant la clé 40. Le coordinator recalcule $\text{hash}(40) \% 5 = 0$ et identifie de nouveau le nœud 8001 comme responsable. Un NODE_GET est transmis au nœud 8001, qui effectue une recherche dans son store local et retourne la valeur 25 via un NODE_RESPONSE. Le coordinator retransmet enfin une CLIENT_RESPONSE(value : 25) correspondant à la valeur associée à la clé 40.

À noter que si la clé recherchée n'existe pas, le coordinator envoie tout de même un NODE_GET au nœud responsable. Ce dernier effectue la recherche dans son store local, ne trouve pas la clé et retourne un NODE_RESPONSE indiquant que la clé est absente. Le coordinator retransmet alors cette information au client sous la forme d'une CLIENT_RESPONSE signalant que la clé n'existe pas.

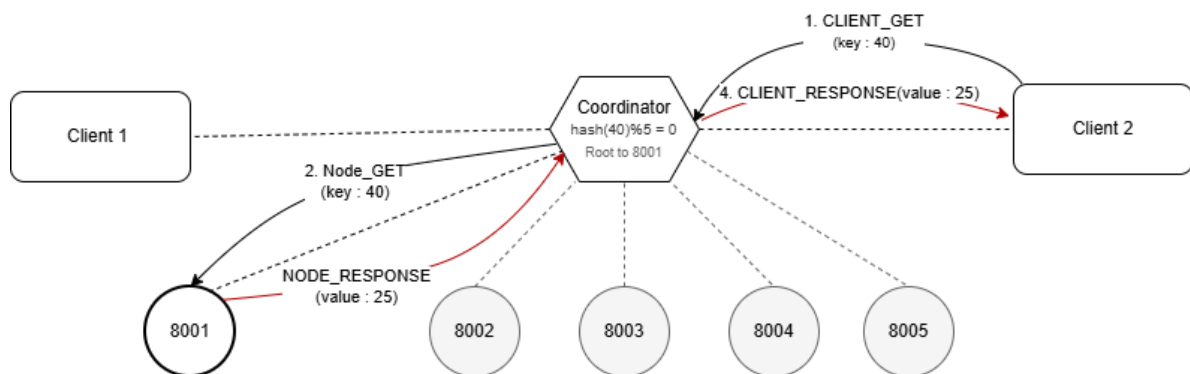


FIGURE 11 – Déroulement d'une opération GET dans l'architecture centralisée.

Nombre total de Message = 4

2.2.3 Opération Delete

La suppression d'une clé suit exactement la même logique que le GET. Le client envoie un CLIENT_DELETE avec la clé de la donnée que le client souhaite supprimer, le coordinator calcule le nœud responsable et lui transmet un NODE_DELETE. Si la clé existe, le nœud supprime la clé de son store local et indique dans sa réponse au coordinator que la clé a bien été supprimé. Si la clé n'existe pas, le nœud envoie au coordinator que la clé n'existe pas. Le coordinator transmet cette réponse reçue au client avec CLIENT_RESPONSE.

Nombre total de Message = 4

2.3 Gestion dynamique des nœuds

La principale problématique avec un système distribué basé sur un calcul hachage modulaire est la valeur N désignant le nombre de nœuds présent dans le réseau. En effet, si un nœud rejoint ou quitte le réseau, cela modifie le résultat du calcul hachage modulaire et donc provoque un échange massif de flux de donnée. En effet, étant donné que le résultat a changé les données stockées doivent migrer vers le bon nœud auquel ils appartiennent après la modification du calcul hachage modulaire.

2.3.1 Intégration d'un nouveau nœud (JOIN)

Lorsqu'un nouveau nœud démarre, il s'enregistre auprès du coordinator en envoyant un message `REGISTER_NODE`. Le coordinator l'ajoute à sa liste de nœuds actifs et déclenche immédiatement un rebalance. En effet, l'ajout d'un nœud fait passer le modulo de N à $N+1$, ce qui modifie le nœud responsable des clés existantes. Le rebalance ne s'agit pas uniquement d'envoyer les données au nouveau venu. En effet, les emplacements de toutes les données sont recalculés et certaines peuvent migrer entre deux nœuds déjà présents sans impliquer le nouveau nœud.

Le rebalance commence par un inventaire complet : le coordinator envoie un `DUMP_REQUEST` à chaque nœud actif, qui lui retourne la liste de ses clés. À partir de cet inventaire, le coordinator recalcule pour chaque clé le nœud qui devrait en être responsable avec le nouveau $N+1$ et construit un plan de migration précis indiquant, pour chaque nœud source, quelles clés doivent être envoyées à quel nœud destination.

Pour chaque entrée de ce plan, le coordinator envoie un message `TRANSFER_KEYS` au nœud source, en lui communiquant la destination et la liste des clés concernées. Le nœud source collecte les valeurs correspondantes dans son store local, les regroupe en un seul message `NODE_BATCH_PUT` et l'envoie directement au nœud destination, sans que les valeurs ne transitent par le coordinator. Le nœud destination stocke le batch et répond `ACK`. Le nœud source supprime alors les clés transférées de son propre store.

Pendant toute la durée de ces transferts, les requêtes clients sont temporairement bloquées. Lorsque le plan de migration contient des déplacements à effectuer, le coordinator envoie au préalable un message `REBALANCE` à tous les nœuds, qui positionnent un flag interne `rebalancing = true`. Toute requête `NODE_PUT`, `NODE_GET` ou `NODE_DELETE` reçue pendant cet état retourne immédiatement un message `REBALANCING` au coordinator, qui le propage au client. Ce blocage garantit qu'aucun client ne tente de lire une clé en cours de déplacement, ce qui éviterait une réponse incorrecte de type `NOT_FOUND` sur une donnée qui existe. À la fin du rebalance, le coordinator diffuse un `REBALANCE_DONE` à tous les nœuds. À la réception de ce message, les nœuds vont positionner leur flag interne `rebalancing = false` pour débloquent les requêtes clients.

2.3.2 Retrait d'un nœud (LEAVE)

Lorsqu'un nœud quitte volontairement le réseau, son `ShutdownHook` envoie un message `UNREGISTER_NODE` au coordinator. La première action du coordinator, avant de retirer ce nœud de sa liste, est d'envoyer un `DUMP_REQUEST` pour récupérer l'intégralité des données qu'il détient. Cette étape est critique : si le dump est effectué après la déconnexion, les données sont définitivement perdues. Une fois les données orphelines récupérées, le coordinator retire le nœud de sa liste et déclenche le rebalance.

Le traitement des données orphelines diffère légèrement du JOIN. Ces données, récupérées par dump avant la déconnexion, sont injectées directement par le coordinator via des messages `NODE_PUT_INTERNAL` vers les nœuds cibles car le nœud source est désormais hors ligne et ne peut plus émettre de `TRANSFER_KEYS`. En revanche, pour les clés déjà présentes sur les nœuds restants qui se retrouvent mal placées suite au changement de modulo de N à $N-1$, le mécanisme est identique au JOIN : le coordinator envoie des `TRANSFER_KEYS` aux nœuds sources, qui transfèrent directement leurs données aux nœuds destinations via `NODE_BATCH_PUT`. Le blocage des clients via le flag `rebalancing` est activé pour les mêmes raisons que lors d'un JOIN.

2.3.3 Détection des pannes par heartbeat

Chaque nœud envoie périodiquement un message HEARTBEAT au coordinator toutes les cinq secondes. Un thread de surveillance vérifie régulièrement ces timestamps : si un nœud n'a pas donné signe de vie depuis plus de quinze secondes, il est considéré comme mort et retiré de la liste des nœuds actifs. Le coordinator déclenche alors un rebalance pour redistribuer les clés selon le nouveau N.

A noter que lors du retrait d'un nœud à cause d'une panne, celui-ci quitte le réseau de manière involontaire. Le nœud étant mort, il est impossible d'effectuer un dump préalable de ses données. Les clés qu'il détenait sont donc définitivement perdues. C'est une des limitations de cette implémentation, qui ne dispose pas de mécanisme de réplication.

2.4 Analyse des performances

2.5 Avantages et limites de l'approche centralisée

L'architecture centralisée présente plusieurs avantages notables. Sa simplicité conceptuelle facilite le débogage et la compréhension du système : un seul composant détient la vue globale du cluster, ce qui rend le routage déterministe et prévisible. La cohérence des données est naturellement garantie puisque toutes les décisions de placement transitent par un unique point de calcul. Le rebalance node-to-node améliore par ailleurs la redistribution en évitant que le coordinator ne transporte les données.

Cependant, cette architecture présente des limites structurelles importantes. Le coordinator constitue un point de défaillance unique. En effet, si le coordinator tombe en panne, l'ensemble du système devient inaccessible. L'absence de réplication représente également une faiblesse majeure : en cas de panne d'un nœud, les données qu'il détenait sont irrémédiablement perdues. Enfin, le hachage modulaire classique entraîne une redistribution massive des données à chaque changement de topologie, comportement qui sera directement comparé à l'approche par hachage cohérent implémentée dans les versions DHT suivantes.

3 Deuxième génération : architecture non structurée inspirée de Gnutella

Cette deuxième implémentation ne repose pas encore sur une vraie DHT structurée au sens de *Chord* ou *Pastry*. Elle correspond plutôt à un réseau pair-à-pair non structuré, inspiré de *Gnutella 0.4*, dans lequel chaque nœud participe à la fois au stockage, au routage et aux échanges réseau. Cette approche a constitué une étape utile pour valider les mécanismes de communication, de découverte et de tolérance aux déconnexions avant l'introduction d'un anneau de hachage cohérent.

3.1 Positionnement général et lien avec Gnutella

Le système suit un modèle *pure P2P* : il n'existe aucune séparation entre clients et serveurs. Chaque processus est un *servent* (*server* + *client*) capable de recevoir des connexions entrantes et d'initier lui-même des connexions sortantes.

Aspect	Implémentation réalisée	Gnutella 0.4
Topologie	Maillage non structuré	Maillage non structuré
Routage	Inondation limitée par TTL	Inondation limitée par TTL
Découverte	PING/PONG	PING/PONG
Recherche	GET	<i>Query</i>
Réponse	REPLY direct	<i>QueryHit</i>
Centralisation	Aucune	Aucune

TABLE 1 – Résumé comparatif entre l'approche naïve développée et les principes de *Gnutella 0.4*.

Cette parenté avec *Gnutella* n'est pas seulement conceptuelle. Elle décrit bien l'intuition de départ : construire d'abord un système entièrement décentralisé, simple à expérimenter, puis observer empiriquement ses limites avant de passer à une DHT structurée.

3.2 Problème initial : l'absence de vue globale

Le premier problème rencontré ne vient pas du transport réseau, mais du placement des données. Dans une topologie trop clairsemée, chaque pair ne connaît qu'un sous-ensemble des autres nœuds. Or si le choix du propriétaire d'une clé dépend de la vue locale du pair qui effectue le calcul, deux machines peuvent aboutir à des décisions différentes pour la même donnée.

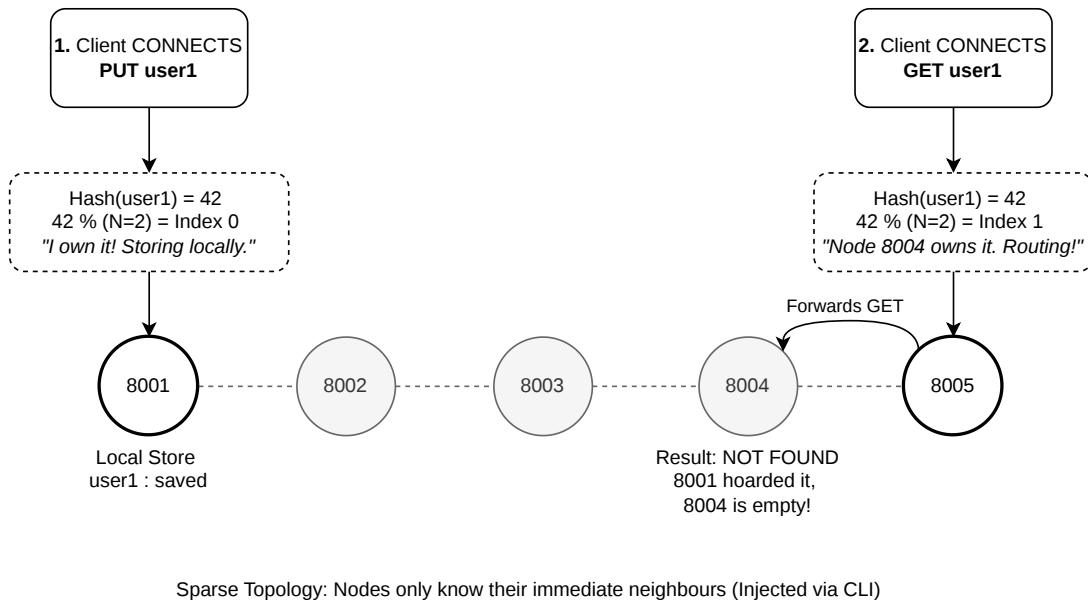


FIGURE 12 – Problème de vue globale dans une topologie clairsemée. Le nœud 8001 stocke localement user1 après avoir calculé $\text{Hash}(\text{user1}) = 42$ avec sa propre vision du système. Plus tard, le nœud 8005 effectue un GET pour la même clé, mais avec une vue réseau différente il conclut que le propriétaire devrait être 8004 et relaie donc la requête dans une mauvaise direction. L'exemple montre qu'en l'absence d'un espace d'adressage partagé et stable, deux nœuds peuvent prendre des décisions incompatibles pour une même clé.

Ce constat a motivé un premier choix d'implémentation : tester explicitement une **structure en mesh** afin d'améliorer la cohérence de la vue réseau. Les schémas présentés dans cette section ne sont donc pas seulement illustratifs ; ils correspondent à des scénarios de test utilisés pour vérifier le comportement du système lorsque tous les nœuds se connaissent, puis lorsqu'un nœud disparaît.

On peut formaliser cette intuition simplement. Si l'on note V l'ensemble des nœuds actifs et E l'ensemble des liens de voisinage, alors un mesh complet correspond au graphe non orienté

$$G = (V, E), \quad E = \{\{u, v\} \mid u, v \in V, u \neq v\}.$$

Pour $|V| = N$, le nombre total d'arêtes vaut alors

$$|E| = \binom{N}{2} = \frac{N(N-1)}{2}.$$

Chaque nœud a donc un degré exactement égal à

$$\deg(v) = N - 1.$$

Cette propriété explique pourquoi la vue locale devient beaucoup plus cohérente : en régime stable, chaque pair voit directement tous les autres.

3.3 Passage à un mesh : ce que cela corrige, et ce que cela casse

Dans une topologie en maillage complet, chaque nœud possède a priori la même vue des participants actifs. Cette organisation réduit donc fortement le problème de vue globale : tant que le réseau reste stable, les pairs appliquent le même calcul de placement et convergent plus facilement vers le même propriétaire logique. Elle corrige aussi partiellement le problème du modulo naïf, car la cohérence de $\mathbb{N}$ est meilleure tant que tous les pairs partagent la même vision du système.

En revanche, cette amélioration reste fragile. Dès qu'un nœud rejoint ou quitte le réseau, la valeur de N change et la fonction de placement $\text{hash}(\text{key}) \% N$ peut désigner un autre propriétaire. Le maillage complet réduit donc l'incohérence à *l'instant donné*, mais ne supprime pas la dépendance structurelle au nombre de nœuds.

Cette fragilité se montre immédiatement par un petit raisonnement. Pour une clé de hachage h , le propriétaire logique dans le schéma naïf est

$$\text{owner}_N(h) = h \bmod N.$$

Après le départ d'un nœud, le système calcule au contraire

$$\text{owner}_{N-1}(h) = h \bmod (N - 1).$$

Les deux valeurs ne coïncident pas en général. Il suffit d'un contre-exemple pour le voir : avec $h = 10$ et $N = 4$, on obtient

$$10 \bmod 4 = 2, \quad 10 \bmod 3 = 1.$$

Le propriétaire logique change donc alors même que la donnée n'a pas physiquement bougé. Plus généralement, l'ensemble des clés qui restent stables après le passage de N à $N - 1$ est

$$S = \{h \in \mathbb{N} \mid h \bmod N = h \bmod (N - 1)\},$$

et cet ensemble est strictement plus petit que l'ensemble de toutes les clés ; il n'existe donc aucune garantie globale de stabilité sous churn.

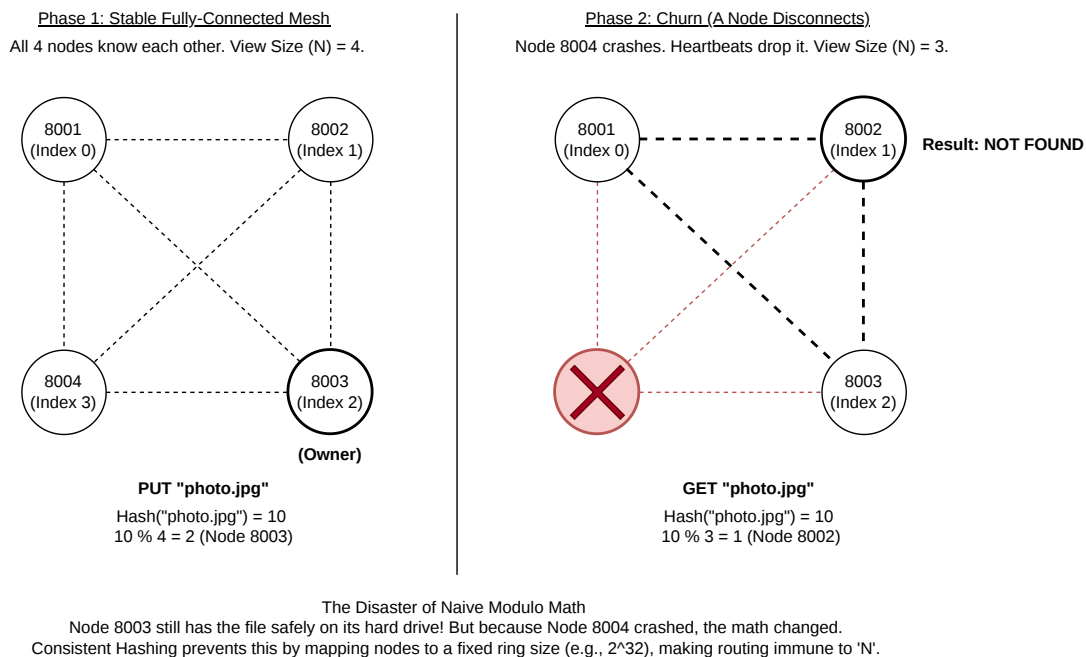


FIGURE 13 – Impact du *churn* sur un placement basé sur $\text{hash}(\text{key}) \% N$. À gauche, avec quatre nœuds connectés en mesh, la clé `photo.jpg` est associée au nœud 8003, qui la stocke localement. À droite, après la panne du nœud 8004, la taille de la vue passe de 4 à 3 ; le même calcul désigne alors 8002 comme propriétaire logique. Le mesh améliore donc la cohérence de la vue tant que le système est stable, mais il ne protège pas contre l'instabilité du placement lorsque N varie.

Autrement dit, la structure en mesh règle en grande partie le problème de vue globale en régime stable, mais elle fait aussi apparaître plus clairement le vrai défaut du schéma naïf : la dépendance du placement au cardinal du réseau. Les tests de churn ont précisément servi à mettre ce point en évidence.

Du point de vue de la complexité, le mesh complet présente aussi un coût structurel élevé. Comme chaque nœud maintient $N - 1$ voisins, la mémoire consacrée aux tables de voisinage est en $\Theta(N)$ par pair, soit $\Theta(N^2)$ à l'échelle du système. De même, un heartbeat envoyé à tous les voisins toutes les $\Delta = 5$ secondes produit environ

$$N(N - 1)$$

messages de contrôle par période si l'on compte chaque émission directionnelle. Le coût de maintenance croît donc quadratiquement avec la taille du réseau.

Cette convergence vers un mesh plus dense n'a toutefois pas été obtenue d'emblée. Dans la pratique, il a fallu faire évoluer le mécanisme de découverte pour que les nœuds convergent rapidement vers une vue commune des participants actifs. Dans la version la plus simple, la vivacité du réseau reposait sur des messages PING/PONG et sur un *heartbeat* périodique : un nœud apprenait surtout l'existence de ses voisins immédiats, puis relayait éventuellement l'information de proche en proche. Ce fonctionnement suffisait pour maintenir un réseau connexe, mais la convergence restait lente et partielle si la topologie de départ était clairsemée.

Un mécanisme que l'on peut décrire comme du *pong caching* ou *peer exchange* a alors été introduit. Lorsqu'un nœud reçoit un PING, il ne répond plus seulement par un PONG attestant de sa propre présence. Il peut aussi renvoyer, dans la charge utile, l'adresse d'un petit nombre d'autres pairs déjà connus localement. Le pair initiateur enrichit alors son propre registre à partir de ces informations sans attendre une nouvelle inondation complète du réseau.

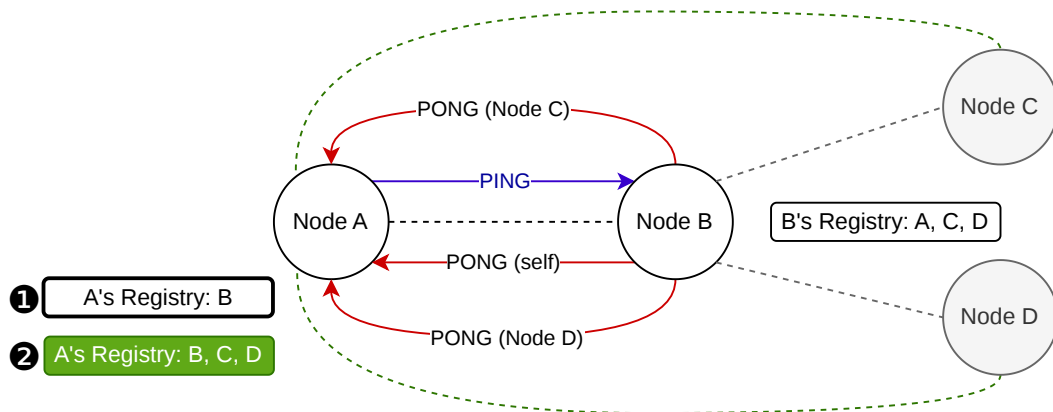


FIGURE 14 – Principe du *pong caching*. En plus d'annoncer sa propre présence, un nœud répondant à un PING peut transmettre quelques adresses supplémentaires issues de son registre local. Par diffusion répétée de ces informations, les pairs convergent plus vite vers une vue commune des participants actifs.

Dans notre contexte de simulation, ce mécanisme produit un effet important : une topologie initialement pauvre converge rapidement vers un maillage beaucoup plus dense, voire quasi complet à petite échelle. Cette convergence améliore à la fois la robustesse et la latence. D'une part, chaque nœud dispose de davantage de chemins alternatifs en cas de panne. D'autre part, le nombre de sauts nécessaires pour atteindre une grande partie des pairs diminue fortement, ce qui rend la diffusion des requêtes plus rapide et plus fiable.

3.4 Le flooding comme correction partielle

Une fois ce problème observé, la question n'était plus seulement "où stocker ?", mais aussi "comment retrouver une donnée quand le calcul local du propriétaire devient faux ou incomplet ?". C'est là que l'inondation limitée joue un rôle utile.

Dans cette version, lorsqu'un nœud reçoit une requête GET et qu'il ne possède pas localement la donnée recherchée, il la retransmet à l'ensemble de ses voisins connus. Le mécanisme est donc une

inondation contrôlée. Ce n'est pas une solution parfaite au problème de placement, mais c'est une **solution de rattrapage** : même si un nœud se trompe sur le propriétaire logique, la requête peut encore atteindre le bon détenteur effectif de la donnée à force de propagation dans le mesh.

- **Diffusion.** Une requête est rediffusée de pair en pair jusqu'à trouver un propriétaire potentiel.
- **TTL.** Un plafond `MAX_HOPS = 10` limite le nombre de sauts pour éviter les boucles infinies et contenir le trafic.
- **Élimination des doublons.** Le `PeerRegistry` mémorise les identifiants déjà vus, par exemple sous la forme `Type|Origin|Seq`, afin d'ignorer immédiatement les messages dupliqués.

On peut borner ce coût de recherche. Dans un graphe de degré moyen d , une inondation de profondeur maximale T génère, dans le pire cas sans élimination efficace des doublons, un nombre de transmissions majoré par une série géométrique :

$$1 + d + d^2 + \dots + d^T = \sum_{i=0}^T d^i.$$

Si $d > 1$, cette somme vaut

$$\sum_{i=0}^T d^i = \frac{d^{T+1} - 1}{d - 1},$$

soit un coût exponentiel en T . Dans un mesh complet, on peut prendre $d = N - 1$, ce qui donne une borne très pessimiste mais révélatrice : la redondance potentielle devient rapidement prohibitive quand N grandit.

Grâce au cache de messages déjà vus, l'implémentation réelle est heureusement plus proche d'un parcours borné des arêtes accessibles que d'une explosion purement exponentielle. Dans ce cas, une requête GET touche au plus chaque lien utile un nombre constant de fois, ce qui ramène l'ordre de grandeur à $O(|E|)$. Pour un mesh complet, cela reste

$$O(|E|) = O\left(\frac{N(N-1)}{2}\right) = O(N^2).$$

Même avec ce filtrage, le pire cas reste donc quadratique en nombre de nœuds. On peut l'écrire plus explicitement dans le cas d'un mesh complet. Comme $|E| = \frac{N(N-1)}{2}$, une recherche GET qui doit potentiellement explorer tout le réseau avant de conclure à un échec, ou avant d'atteindre le seul nœud possédant la donnée en dernier, effectue dans le pire cas un nombre de transmissions proportionnel à

$$T_{\text{worst}}(N) \approx 2|E| = N(N-1).$$

Le facteur 2 vient du fait qu'un lien bidirectionnel peut être sollicité dans les deux sens au cours de la propagation et des relais. On obtient donc

$$T_{\text{worst}}(N) \in \Theta(N^2).$$

Autrement dit, doubler le nombre de nœuds multiplie approximativement par quatre le coût maximal d'une recherche exhaustive, ce qui confirme que cette stratégie ne passe pas bien à l'échelle.

Ces bornes montrent bien l'ambivalence du flooding dans cette architecture naïve. D'un côté, il constitue un mécanisme de rattrapage robuste : si le réseau reste connexe, qu'aucun lien critique n'échoue pendant la recherche, et que le TTL est au moins égal au diamètre D du graphe, alors une requête émise depuis un nœud source finit nécessairement par atteindre tout nœud cible détenant la donnée. Dans un mesh complet, le diamètre vaut

$$D = 1,$$

ce qui explique pourquoi le flooding y est particulièrement efficace : tant que le système reste stable, la découverte est presque immédiate.

D'un autre côté, cette propriété se paie cher. Le flooding ne résout le problème que partiellement : il compense une mauvaise localisation par davantage de trafic réseau. Le système devient donc plus tolérant aux incohérences, mais au prix d'un coût en messages qui croît vite avec la taille du réseau. Cette tension entre *robustesse de découverte* et *coût quadratique ou pire* résume précisément la limite de l'approche naïve.

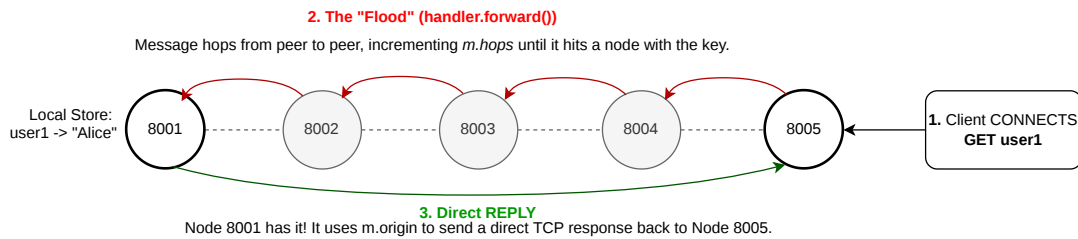


FIGURE 15 – Recherche par inondation dans l'approche naïve. Le nœud 8005 émet une requête GET `user1` ; celle-ci est relayée de proche en proche par les pairs intermédiaires, avec incrément du compteur de sauts, jusqu'au nœud 8001 qui possède la donnée dans son stockage local. Dans ce sens, le flooding corrige partiellement les erreurs de placement ou de vue locale : même si le demandeur ne connaît pas le bon propriétaire, la propagation augmente ses chances d'atteindre le nœud qui détient effectivement la donnée.

3.5 Réponse directe, puis *reverse path caching*

Une fois le mécanisme de flooding validé, la manière dont les réponses revenaient vers le demandeur a ensuite été modifiée. L'idée n'a pas été de transformer la recherche elle-même, mais plutôt de partir des limites de la réponse directe pour introduire une variante fondée sur la réutilisation du chemin de retour.

Dans la première version, lorsqu'un nœud trouvait la clé demandée après propagation d'une requête GET, il renvoyait la valeur directement au demandeur en ouvrant une connexion TCP dédiée grâce au champ `origin`. Ce choix était simple et plutôt efficace à petite échelle : la recherche restait diffuse, mais la réponse suivait un chemin direct.

Cette stratégie présentait toutefois une limite importante. Si une donnée devenait populaire, le nœud qui la détenait restait le seul à répondre à toutes les requêtes. La charge réseau et la charge de service se concentraient donc sur un unique point, tandis que les nœuds intermédiaires ne faisaient que relayer la recherche sans rien apprendre des données traversant le réseau. En pratique, le système restait fonctionnel, mais il ne tirait aucun bénéfice de la répétition des requêtes.

C'est précisément cette limite qui a conduit à l'introduction d'une deuxième version du mécanisme de réponse, fondée sur le *reverse path caching*. Au lieu de laisser le nœud propriétaire répondre directement au client par un nouveau canal, le chemin effectivement parcouru par la requête GET a été réutilisé.

Lors de la propagation de la requête, chaque nœud intermédiaire enregistre temporairement de quel voisin elle provient. Concrètement, une table de routage locale associe un identifiant de route, par exemple `seq:key`, à l'adresse du saut précédent. Ces entrées jouent le rôle de *breadcrumbs* : elles ne décrivent pas toute la topologie, mais elles suffisent à reconstruire un chemin de retour déterministe pour la réponse.

Lorsque le nœud propriétaire produit le message REPLY, celui-ci remonte alors exactement ce chemin inverse. À chaque saut, le nœud intermédiaire lit l'entrée correspondante, la supprime pour libérer la mémoire, met en cache localement la paire (`key`, `value`), puis transmet la réponse au saut précédent. La réponse ne suit donc plus un chemin direct indépendant ; elle exploite désormais la trace laissée par la requête initiale.

Cette modification change sensiblement le comportement du système, et la Figure 16 permet de le voir pas à pas. Lors du premier GET, le réseau est encore dans une situation de *cache miss* : le `lookup` rouge doit aller jusqu'au détenteur réel de la donnée, puis la réponse verte remonte tout le chemin inverse. C'est précisément pendant cette phase de retour que le cache se construit, puisque chaque nœud intermédiaire conserve une copie de la valeur avant de relayer le REPLY. Lors d'une requête ultérieure, on bascule alors vers un *cache hit* : le `lookup` peut être intercepté plus tôt, la réponse part depuis un nœud plus proche, et le chemin total devient plus court. Autrement dit, on passe d'une

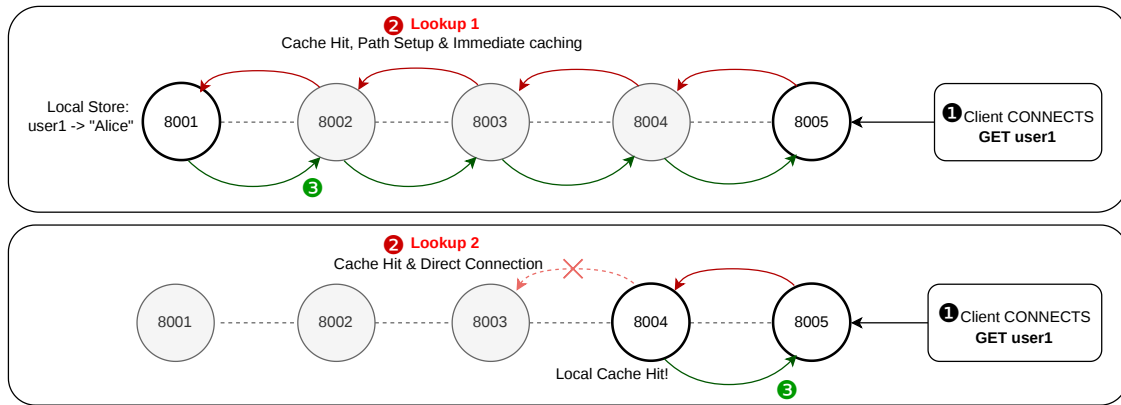


FIGURE 16 – Lecture détaillée du *reverse path caching*. Dans la partie haute (*Lookup 1*), le chemin rouge correspond au *lookup* initial émis par le client connecté à 8005 pour obtenir *user1*. La donnée n'étant pas présente localement, il s'agit d'un *cache miss* : la requête progresse de pair en pair jusqu'au nœud 8001, qui possède la valeur dans son stockage local. Le chemin vert représente ensuite la réponse *REPLY* qui remonte en sens inverse ; à chaque saut, les nœuds intermédiaires utilisent les *breadcrumbs* laissés par le *lookup* pour relayer la réponse et mettre immédiatement la donnée en cache. Dans la partie basse (*Lookup 2*), une nouvelle requête pour la même clé repart de 8005, mais elle rencontre cette fois un *cache hit* sur un nœud intermédiaire proche, ici 8004. Le *lookup* rouge est alors plus court, et la réponse verte revient directement sans devoir remonter jusqu'au propriétaire initial 8001. Le schéma montre ainsi à la fois le premier *cache miss*, la phase d'amorçage du cache le long du chemin inverse, puis le second *cache hit* qui réduit la distance de résolution.

logique où chaque demande repart presque de zéro à une logique d'apprentissage progressif du réseau. La recherche reste fondée sur le flooding, mais son coût effectif peut diminuer fortement pour les données populaires grâce à cette mise en cache opportuniste.

3.6 Usages réels et intérêt pratique

Il serait toutefois réducteur de conclure que ce type d'architecture n'a aucun intérêt pratique. Sous sa forme brute, un réseau entièrement de type Gnutella passe mal à l'échelle pour un stockage distribué global, précisément à cause du coût du flooding et de l'instabilité du placement. En revanche, ses idées restent très utiles dans des contextes où la simplicité, la robustesse et la découverte rapide priment sur l'optimalité asymptotique.

On retrouve aujourd'hui ces idées dans des briques très concrètes.

Sur un réseau local, **mDNS** (*multicast DNS*) est un mécanisme de découverte sans infrastructure (<https://datatracker.ietf.org/doc/html/rfc6762>). Son rôle est simple : permettre à des machines présentes sur le même réseau de s'annoncer mutuellement et de découvrir rapidement leurs voisins sans passer par un serveur central. L'idée est très proche de notre approche naïve : on privilégie la simplicité, la visibilité rapide et l'absence de coordination centrale.

Dans l'écosystème **libp2p**, on retrouve également cette logique pour la découverte locale des pairs (<https://libp2p.io/docs/mdns/>). Un nœud peut ainsi détecter d'autres participants proches et établir rapidement ses premières connexions. À un autre niveau, les protocoles *Floodsub* puis surtout *Gossipsub* servent à diffuser des messages dans le réseau pair-à-pair (<https://libp2p.io/docs/pubsub/>). Ils reprennent l'intuition du flooding, mais sous une forme mieux contrôlée afin de limiter le trafic inutile.

Dans les **blockchains modernes**, la couche réseau pair-à-pair conserve elle aussi cette idée de propagation rapide (<https://ethereum.org/developers/docs/networking-layer/>). Concrètement, les nœuds commencent par découvrir d'autres pairs, puis utilisent un mécanisme de type gossip pour transmettre transactions et blocs à travers le réseau. L'objectif n'est pas de stocker les données via

une DHT naïve, mais de propager vite et sans point central une information qui doit atteindre un grand nombre de participants.

Autrement dit, l'approche naïve inspirée de Gnutella reste pertinente comme brique de base pour des tâches bien ciblées : trouver rapidement des voisins, diffuser une information à grande vitesse, ou maintenir une connectivité décentralisée sans coordinateur central. Ce ne sont pas les mêmes objectifs qu'une DHT structurée ; c'est précisément pourquoi, dans les systèmes réels, ces mécanismes coexistent souvent avec des structures plus élaborées au lieu de les remplacer entièrement.

3.7 Bilan

Cette deuxième génération a permis de valider plusieurs briques essentielles : communication TCP fiable, sérialisation des messages, gestion des connexions concurrentes, découverte des pairs, surveillance de la vivacité et recherche distribuée. Elle a surtout montré une progression expérimentale claire : d'abord, le mécanisme de découverte a permis au réseau de converger vers un mesh beaucoup plus cohérent ; ensuite, le *reverse path caching* a permis de réutiliser les chemins de retour pour diffuser des copies utiles des données.

Du point de vue des performances, cette architecture a toutefois un comportement très contrasté. Dans le meilleur cas, si la donnée est locale ou si le bon pair est immédiatement accessible, une requête GET se résout en pratique en $O(1)$ sauts. Dans le pire cas, au contraire, la recherche doit explorer une grande partie du réseau ; sur un mesh complet, son coût atteint alors $\Theta(N^2)$ en nombre de transmissions. De la même manière, la topologie complète offre un diamètre minimal égal à 1, ce qui maximise la rapidité de découverte, mais son coût structurel reste quadratique : chaque pair maintient $N - 1$ voisins, soit une mémoire en $\Theta(N)$ localement et en $\Theta(N^2)$ à l'échelle du système.

Le rôle de la topologie est donc central. Une topologie clairsemée réduit les coûts de maintenance, mais elle produit des vues locales divergentes et donc des décisions incompatibles sur le placement des clés. À l'inverse, un mesh complet améliore fortement la cohérence de la vue réseau, mais il ne corrige pas le problème fondamental du placement par $\text{hash}(\text{key}) \% N$: dès que N varie, le propriétaire logique d'une clé peut changer. On obtient donc un compromis défavorable entre cohérence locale, coût de maintenance et stabilité globale du placement.

En revanche, pour le placement stable des données et le routage scalable, ces améliorations restent palliatives. Le mesh ne supprime pas la dépendance à N , et le flooding compense l'erreur par un surcoût en messages. Ces limites motivent directement l'introduction de vraies DHT structurées dans la suite du rapport. Des approches comme *Chord* ou *Pastry* remplacent la dépendance à N par un espace d'identifiants fixe et introduisent des mécanismes de routage déterministes, beaucoup mieux adaptés à un environnement distribué dynamique.

4 Troisième génération : DHT structurée inspirée de Chord

4.1 Vue d'ensemble de l'architecture

Cette troisième implémentation adopte une architecture DHT structurée inspirée de Chord. Contrairement aux générations précédentes, il n'existe ni *coordinator* central ni *flooding* : chaque nœud s'auto-organise dans un anneau logique et route de manière déterministe les requêtes vers le nœud responsable, sans connaissance globale du réseau.

Cette troisième implémentation repose sur deux concepts fondamentaux :

- Un **anneau logique** : chaque nœud occupe une position fixe sur un anneau de taille $2^6 = 64$, calculée par hachage de son adresse. Les clés sont distribuées sur cet anneau via *consistent hashing*.
- Une **finger table** : table de routage locale de $M = 6$ entrées permettant à chaque nœud de router les requêtes en $O(\log N)$ messages sans connaissance globale du réseau.

La cohérence de l'anneau est assurée par un service de stabilisation périodique qui corrige les pointeurs *successor* et *predecessor* et maintient une liste de successeurs de secours pour la tolérance aux pannes.

4.1.1 Principe de fonctionnement

Chaque nœud et chaque clé reçoivent un identifiant sur m bits (souvent via SHA-1), placé sur un anneau modulo 2^m . Une clé k est stockée sur son *successor* : le premier nœud rencontré dans le sens horaire dont l'identifiant est supérieur ou à k .

Cette règle donne trois propriétés utiles :

- **Répartition** : le *consistent hashing* distribue les clés de manière quasi uniforme.
- **Scalabilité** : l'ajout/retrait d'un nœud ne déplace qu'une fraction des clés contrairement au $\text{hash}(\text{key}) \% N$ des générations précédentes.
- **Décentralisation** : aucun serveur central n'est requis.

4.1.2 Routage efficace : Finger Table

Pour éviter un parcours linéaire de l'anneau, chaque nœud maintient une **finger table** de m entrées. L'entrée i pointe vers le premier nœud joignable à partir de $n + 2^{i-1}$.

$$\text{finger}[i].\text{start} = (n + 2^{i-1}) \bmod 2^m$$

Cette structure permet des sauts exponentiels et un coût moyen de recherche en $O(\log N)$ messages : le routage consiste à sélectionner le finger le plus proche précédant la clé cible (*closestPrecedingFinger*), et à déléguer la requête à ce nœud, jusqu'à atteindre le *successor* de la clé.

4.1.3 Opération PUT

Lorsqu'un nœud reçoit une requête PUT pour une paire $(\text{key}, \text{value})$, il calcule le *hash* de la clé et appelle *lookup* pour identifier le *successor* responsable. Si ce *successor* est le nœud lui-même, la paire est stockée localement. Sinon, la requête est routée vers le nœud désigné via la *finger table*. Le message transite ainsi de nœud en nœud jusqu'à atteindre le responsable, qui effectue le stockage final.

Nombre total de messages échangés : $O(\log N)$

4.1.4 Opération GET

La lecture suit exactement la même logique que le PUT. Le nœud recevant la requête GET calcule le *hash* de la clé et identifie le *successor* responsable via *lookup*. Si c'est lui-même, il retourne la valeur depuis son *store* local, ou NOT_FOUND si la clé est absente. Sinon, la requête est routée vers le nœud responsable, qui répond directement à l'émetteur original.

Nombre total de messages échangés : $O(\log N)$

4.1.5 Intégration d'un nouveau nœud (JOIN)

Lorsqu'un nouveau nœud démarre, il contacte un nœud existant passé en paramètre (*bootstrapPeer*) et appelle *remoteCallFindSuccessor* pour identifier sa position sur l'anneau. Une fois son *successor* déterminé, il lui envoie un message REQUEST_KEYS pour récupérer les clés dont il devient responsable. Contrairement à la première génération, ce transfert est ciblé : seules les clés appartenant à la plage (prédécesseur du successor, id du nouveau nœud] sont déplacées. Le service de stabilisation se charge ensuite de propager progressivement la connaissance du nouveau nœud aux autres membres de l'anneau via *notify*.

4.1.6 Retrait d'un nœud (LEAVE)

Lors d'un départ volontaire, le *ShutdownHook* déclenche la procédure de sortie propre. Le nœud transfère d'abord l'intégralité de son *store* local à son *successor* via *transferAllKeysTo*. Il envoie ensuite un message LEAVE_NOTIFY à son *successor* et à son *predecessor* pour qu'ils mettent à jour leurs pointeurs et maintiennent la cohérence de l'anneau.

4.1.7 Détection des pannes et stabilisation

Le service de stabilisation (*StabilizeService*) s'exécute toutes les 2 secondes. À chaque cycle, il vérifie la vivacité du *successor* via *remoteCallGetPredecessor* : si celui-ci ne répond pas, il cherche un remplaçant dans la liste de successeurs de secours, puis dans la *finger table*. Le *predecessor* est vérifié tous les 3 cycles. En cas de panne, il est remis à *null* et l'anneau se reconfigure progressivement. La *finger table* est quant à elle mise à jour entrée par entrée à chaque cycle via *fixFingers*.

4.1.8 Avantages et limites de l'approche Chord

L'architecture Chord présente plusieurs avantages majeurs par rapport aux générations précédentes. Le *consistent hashing* garantit une distribution quasi uniforme des clés et limite les transferts lors des changements de topologie : un JOIN ou LEAVE ne déplace qu'une fraction des clés, contre une redistribution massive dans la première génération. Le routage déterministe via la *finger table* offre un *lookup* en $O(\log N)$ messages, sans *flooding* ni *coordinator* central. Enfin, la liste de successeurs de secours et la stabilisation périodique assurent une tolérance aux pannes que les générations précédentes ne permettaient pas.

Cependant, cette architecture présente également des limites. La stabilisation périodique introduit un délai de convergence : dans une période de *churn* intense, l'anneau peut temporairement être dans un état incohérent, rendant certaines clés introuvables le temps que les pointeurs se corrigent. Par ailleurs, notre implémentation utilise un espace d'adressage de taille $2^6 = 64$, ce qui limite le nombre de nœuds distincts pouvant coexister sur l'anneau. Enfin, comme dans la première génération, l'absence de réplication implique qu'une panne non détectée avant la stabilisation peut entraîner une perte temporaire de données.

5 Analyse des métriques